İçindekiler

Prompt Kütüphanesi (Türkçe) **1**
Bölüm 0 — Kullanım . . . 2

Bölüm 1 — Temel promptlar **2**
1. Oturum başlangıcı . . . 2
2. Koddan önce plan . . . 2
3. API endpoint tasarımı . . . 3
4. Concurrency incelemesi . . . 3
5. Güvenlik incelemesi . . . 3
6. Testler . . . 4
7. ADR taslağı . . . 4
8. Ders kaydetme . . . 4
9. Migration kaydetme . . . 4
10. Standard güncelleme . . . 5
11. SDD'yi dürüst tutma . . . 5
12. Diff incelemesi . . . 5
13. Yük testi hazırlığı . . . 5
14. Savunma provası . . . 5
15. Brain sağlık kontrolü . . . 5
16. Durum sorgusu — kurulu mu? . . . 6

Bölüm 2 — Kesintisiz çalıştırma **6**
2.1 Döngü . . . 6
2.2 Aşama 1 — sıradaki işi seç . . . 7
2.3 Aşama 2 — spec yaz . . . 7
2.4 Aşama 3 — uygula . . . 8
2.5 Aşama 4 — doğrula ve kendi kendini tamir et . . . 8
2.6 Aşama 5 — kapanış . . . 9
2.7 Durma koşulları . . . 9
2.8 Her insansız koşuya eklenecek guardrail bloğu . . . 9
2.9 Durduktan sonra devam . . . 10

Bölüm 3 — Araca özel **10**
3.1 Claude Code . . . 10
3.2 Gemini CLI . . . 11
3.3 Cursor, Copilot, Aider, Codex . . . 11
3.4 CI içinden herhangi bir ajan . . . 11

Bölüm 4 — Snippet'lar **11**
4.1 Araç bağımsız sürücü . . . 11
4.2 Ajanın geçmesi gereken yerel kapı . . . 12
4.3 Sonuç envelope'u . . . 12
4.4 Spec durum taraması . . . 12
4.5 Gecelik Brain sağlık kontrolü . . . 12

Bölüm 5 — Anti-pattern'ler **12**
İlgili . . . 13

Prompt Kütüphanesi (Türkçe)

`music-room` üzerinde AI kodlama ajanıyla çalışmak için promptlar.

- **Bölüm 1** ajan bağımsız. Her blok Claude Code, Gemini CLI, Cursor, Copilot, Aider, Codex veya düz API çağrısında çalışır.
- **Bölüm 2** kesintisiz çalıştırma: spec döngüsü, kendi kendini tamir eden build, durma koşulları, guardrail'ler.
- **Bölüm 3** araca özel ekler, başka araçta karşılığı olmayan Claude Code özellikleri dahil.
- **Bölüm 4** tutkal: script'ler, envelope, CI kancaları.

Kanonik sürüm İngilizce: [[prompt-library]]. Bu dosya okuma kolaylığı içindir. İkisi çeliştiğinde İngilizce geçerlidir. Prompt gövdeleri Türkçeye çevrildi; **dosya yolları, kural kimlikleri, komutlar ve `RESULT:` satırı birebir korundu** — ajan onları arıyor.

Bölüm 0 — Kullanım

Her ajanın önce okuduğu sözleşme

Kök dizindeki `AGENTS.md`. Claude Code ek olarak `CLAUDE.md`, Gemini CLI `GEMINI.md` okuyor; ikisi de `AGENTS.md`'ye işaret ediyor. Ortak sözleşme tek yerde.

Aşağıdaki promptların kısa olma sebebi bu: kurallar zaten yüklü. Projeyi baştan anlatan bir prompt, projeden sapacak bir prompttur.

Buraya ekleme yaparken taşınabilirlik kuralları

1. **Bölüm 1'de araca özel sözdizimi yok.** Slash komutu yok, `@dosya` yok, araç adı yok.
2. **Dosyalara depo göreli yolla referans ver.** `docs/owasp/TOP10-CHECKLIST.md`, "şu OWASP dokümanı" değil.
3. **Düzenlemeden önce dosya listesi iste.** Her yerde çalışır ve ajanı kapsam içinde tutmanın tek güvenilir yolu.
4. **Somut çıktı iste**, his değil. "Bulgu başına tek satır path:line" > "dikkatlice incele".
5. **Durma koşulunu söyle.** Sınırsız prompt sınırsız diff üretir.

Bölüm 1 — Temel promptlar

1. Oturum başlangıcı

```
brain/MEMORY.md dosyasını oku ve router'ını takip et. Workspace'i tarama.
Sonra 10 satırı geçmeden söyle: güncel scope, kararlaştırılmış stack, ve açık maddeler.
```

Dar varyant

```
Önce brain/MEMORY.md, sonra sadece brain/projects/musicroom-backend.md dosyasını oku.
Backend için neyin kararlaştırıldığını ve neyin açık olduğunu özetle.
Başka hiçbir dosyayı okuma.
```

2. Koddan önce plan

```
Görev: <ne istiyorsan>.

Kod yazmadan önce:
1. brain/projects/musicroom-backend.md ve ilgili brain/architecture/ notunu oku.
2. brain/standards-router.md üzerinden ilgili standartları oku.
3. Kontrat için docs/api/ENDPOINTS.md ve docs/owasp/API-TOP10-CHECKLIST.md dosyalarını oku.
4. Oluşturacağın veya değiştireceğin her dosyayı MusicRoom/ altındaki yoluyla listele.
```

```
Sonra dur ve onayımı bekle. Bu turda kod yazma.

Kısıtlar:
- Backend kodu MusicRoom/, client kodu MusicRoomApp/ altına. reference/ salt okunur.
- Use case başına bir command veya query; handler ince kalır.
- Kaynak kodda secret yok; config .env üzerinden.
- .editorconfig limitleri: metot 40 satır, complexity 10, 5 parametre, tip başına 15 metot.
```

Onaydan sonra

```
Listelediğin planı birebir uygula. O listenin dışındaki hiçbir dosyayı değiştirme.
Dokunduğun katmanlar için testing standardının gerektirdiği testleri ekle.
Bitince değişen dosyaları ve doğrulama komutlarını yazdır.
```

3. API endpoint tasarımı

```
<bağlam> için REST endpoint'lerini tasarla; referans:
docs/technical-guide/20-api/rest-api-design/STANDARD.md ve docs/api/ENDPOINTS.md içindeki mevcut
şekil.

Her endpoint için: method, path, request body, response body, dönebileceği TÜM status kodları,
ve iki yetkilendirme kapısı (ownership, ve visibility/licence).

Gizli kaynağın 403 yerine 404 dönmesi gereken her endpoint'i işaretle.
Kod yazma.
```

4. Concurrency incelemesi

Subject race yönetimini dört yerde notlandırıyor.

```
<dosya veya özellik> için race condition incelemesi yap.
Bağlam: brain/architecture/realtime-architecture.md ve docs/SDD.md bölüm 12.

Özellikle kontrol et:
- iki eşzamanlı istek aynı kullanıcıdan aynı parçaya iki oy üretebilir mi?
- iki eşzamanlı reorder playlist sırasını tutarsız bırakabilir mi?
- iki client aynı playback session'ı ilerletip bir parçayı atlatabilir mi?
- iki cihaz aynı anda kontrolü tutabilir mi?
- her garanti bir database constraint ile mi sağlanıyor, yoksa sadece uygulama kontrolüyle mi?
- transaction commit olmadan önce broadcast yapılan bir yer var mı?

Her bulgu için onu bozan somut interleaving, sonra çözüm.
Somut interleaving olarak anlatamadığın hiçbir şeyi rapor etme.
```

5. Güvenlik incelemesi

```
<kapsam> için güvenlik incelemesi yap, bu sırayla:
- docs/owasp/TOP10-CHECKLIST.md
- docs/owasp/API-TOP10-CHECKLIST.md
- docs/owasp/ASVS-CONFORMANCE.md (sadece commit edilmiş bölümler)
- docs/technical-guide/40-security/
- brain/constraints/subject-requirements.md bölüm V.6

Bulgu başına tek satır: path:line, severity, checklist kimliği (A01 / API1 / ASVS requirement id),
```

```
problem, çözüm.

Özellikle: her endpoint VE her hub group join için object-level authorisation; per-field profile
filtreleme; credential endpoint'lerinde rate limit; aud ve nonce dahil ID token doğrulaması;
BCrypt cost ve 72 byte giriş limiti; loglanmaması gereken her şey.

Rapor ettiğin ama düzeltmediğin her bulgu için docs/owasp/THREAT-REGISTER.md satırını üret.

Övgü yok. Sorunsuz kod hakkında hiçbir şey söyleme.
```

6. Testler

```
<özellik> için docs/technical-guide/60-quality/testing/STANDARD.md dosyasına uygun test yaz.

Katman: <domain unit | application unit | persistence integration | API integration | realtime>.
xUnit + NSubstitute. Persistence testleri Testcontainers ile gerçek PostgreSQL'e karşı,
asla in-memory provider ile değil.

Her testi Method_Scenario_ExpectedResult şeklinde adlandır.
Negatif yetkilendirme senaryosunu dahil et.
Bir concurrency hotspot'a dokunuyorsa, işlemi paralel çalıştırıp final database state'ini
doğrulayan bir test ekle. Sıralı test race hakkında hiçbir şey kanıtlamaz.
```

7. ADR taslağı

Kararı sen verirsin. Ajan kaydeder.

```
Şu kararı verdik: <karar>.

brain/decisions/ADR-XXXX-<slug>.md dosyasını mevcut ADR'lerin şeklinde oluştur:
Context, Decision, Consequences, Alternatives rejected.
Numarayı brain/decisions/decisions-index.md içindeki en yüksekten devam ettir.

Sonra yay:
- decisions-index.md satırını ekle
- bu kararın geçersiz kıldığı project card veya architecture notunu güncelle
- docs/SDD.md ilgili bölümünü, bölüm 16 ve 17 tablolarını güncelle
- etkilenen standardı güncelle, sürümünü bump et, PDF'ini yeniden üret

Consequences'ta faydayı değil maliyeti de yaz. Bunu ben savunacağım.
```

8. Ders kaydetme

```
brain/lessons/lessons-index.md dosyasına satır ekle: <öğrendiğimiz şey>.
Tek cümlelik kalıcı olgu + kaynağı. Transcript yok, log yok, debug anlatısı yok.
Sadece bu hafta geçerliyse ekleme.
```

9. Migration kaydetme

```
<isim> migration'ını oluşturdum.
brain/migrations/migrations-index.md satırını ekle: tip, geri alınabilir mi, destructive ise
rollback yolu. Referans:
docs/technical-guide/30-data/database-design-migrations/STANDARD.md
```

10. Standard güncelleme

```
docs/technical-guide/<yol>/STANDARD.md dosyasını güncelle: <değişiklik>.

Kurallar:
- Official requirements ile Project policy ayrı kalsın; takım kararını official bölüme taşıma
- Başka standartta olan kuralı tekrarlama; referans ver
- frontmatter sürümünü bump et: ifade patch, yeni kural minor, mevcut kodu geçersiz kılan major
- PDF'i docs/build-pdf.sh ile yeniden üret
- docs/technical-guide/manifest.json dosyasını yeniden üret
```

11. SDD'yi dürüst tutma

```
<karar veya tasarım değişikliği> tamamlandı.

docs/SDD.md dosyasını güncelle: etkilenen viewpoint bölümü, bölüm 16 traceability tablosu, ve bir
madde açtıysa/kapattıysa bölüm 17 tabloları.

Kurallar:
- ADR gerekçesini SDD'de tekrar etme; ADR'ye referans ver
- kodu olmayan hiçbir şey için bölüm 16'da "done" yazma
- karar gerçekten eksikse UNKNOWN - requires decision olarak bırak
- PDF'i docs/build-pdf.sh ile yeniden üret
```

12. Diff incelemesi

```
Mevcut diff'i, dokunduğu dosyalara uygulanan standartlara karşı incele.
Bulgu başına tek satır: path:line: severity: problem. çözüm.
Severity'ye göre sırala. Formatlama detaylarını atla; dotnet format zaten kontrol ediyor.
Sorunsuz kod hakkında hiçbir şey söyleme.
```

13. Yük testi hazırlığı

```
<senaryo> için Gatling simulation'ını hazırla; referans:
docs/technical-guide/60-quality/load-testing/STANDARD.md

Simulation dosyasını, tam komutu ve concurrency rampasını ver. Çalıştırmadan önce kaydetmem
gereken makine bilgilerini hatırlat: sunucu özelliği beyan edilmeden rapor geçersizdir.
REST throughput ve SignalR connection fan-out'u ayrı ölç.
```

14. Savunma provası

```
Bu projeyi savunacağım. Beni <alan> konusunda sorgula.
Sorularını brain/constraints/subject-requirements.md ve brain/decisions/ içeriğinden üret.
Bir peer evaluator'ın soracağını sor, "neden <elenen alternatif> değil" dahil.
Ben denemeden cevabı verme.
```

15. Brain sağlık kontrolü

```
Kaynak kodu okumadan bilgi katmanını denetle:
- birbiriyle çelişen brain/ notları
- kapanmış bir ADR'nin zaten cevapladığı UNKNOWN kayıtları
- çözülmeyen wikilink'ler
- brain/standards-router.md içinde listelenip var olmayan standartlar, veya tersi
- brain/ içinde adı geçip artık var olmayan dosya, fonksiyon, flag
```

```
- docs/SDD.md bölüm 16'da kodun desteklemediğini iddia eden satırlar

Sadece gerçek uyuşmazlıkları raporla. İş uydurma.
```

16. Durum sorgusu — kurulu mu?

Salt okunur. Prompt 2'den önce çalıştır; böylece plan, dokümanların iddiasından değil kodun yaptığından başlar.

```
brain/MEMORY.md dosyasını oku ve router'ını takip et. Workspace'i tarama.

Soru: <X> uygulanmış mı?

Sadece kanıta dayanarak cevapla:
- onu uygulayan tipler, endpoint'ler, handler'lar veya migration'lar; MusicRoom/ veya
  MusicRoomApp/ altında path:line olarak
- onu kapsayan testler, path:line olarak
- brain/decisions/decisions-index.md içinden kararı veren ADR; id'si ve frontmatter status'ü ile
- docs/SDD.md bölüm 16, docs/DELIVERY-PLAN.md ve docs/api/ENDPOINTS.md bu konuda ne iddia ediyor
- bölüm 16 sözlüğüyle tek bir hüküm: done / partly implemented / designed / absent /
  claimed-but-absent

"designed" = Accepted bir ADR var, kod yok. Burada normal bir durum, eksik değil.
"absent" = ne ADR var ne kod.
"claimed-but-absent" = bölüm 16 done veya partly implemented diyor, kod bunu desteklemiyor.
Satırı ve kodu yan yana alıntıla.
İş önerme, kod yazma, dosya değiştirme.
```

Dar varyant — tek kural veya tek gate

```
MusicRoom/ <kural> kuralını zorluyor mu?

Üç şey ver: zorlayan kod path:line olarak, bunu kanıtlayan test path:line olarak, ve eşlendiği
docs/owasp/API-TOP10-CHECKLIST.md satırı.
Üçünden biri eksikse cevap "not enforced" - hangisinin eksik olduğunu söyle.
Kod yazma.
```

Bölüm 2 — Kesintisiz çalıştırma

> **Önce bunu oku.** Subject bölüm II net: entelektüel liderlik takımda kalır ve her tasarım kararı savunulabilmelidir. Buradaki otonomi bu yüzden **zaten onaylanmış kararların içinde kalan spec'leri uygulamakla** sınırlı. Ajan mimari seçmez, bağımlılık eklemez, güvenlik yüzeyine insansız dokunmaz. `docs/specs/README.md` içindeki gate bu farkı temenni olmaktan çıkarıp zorunlu hale getiren şey.

2.1 Döngü

```
sıradaki işi seç -> spec yaz -> gate -> onaylı mı -> uygula -> doğrula
                                  |        hayır                 |
                                  +--> escalate, dur             |
                                                    başarısız +--> kendi kendini tamir (max 3)
                                                                  |
                                                      hâlâ hatalı +--> dur, raporla
```

Her aşama tek prompt. Ayrı olmalarının sebebi: bir koşu durdurulabilsin, incelenebilsin, kesilebilsin.

2.2 Aşama 1 — sıradaki işi seç

Seçmeden önce devam et. İnsanın sonradan onayladığı blocked bir spec sıradaki iştir; aynı iş için ikinci bir spec yazmak onayı çöpe atar ve kaydı ikiye böler.

```
Önce: docs/specs/ dizinini listele ve her SPEC-*.md dosyasının frontmatter'ını oku.

Herhangi birinin status: approved ise, en küçük numaralı olanı al, şunu yazdır

  RESUME: <spec yolu>

ve doğrudan aşama 3'e geç. Yeni spec yazma. Gate'i tekrar çalıştırma - insan zaten cevapladı,
cevapları escalation bloğunda. Onları oku; bağlayıcıdır.

Aksi halde yeni iş seç:

brain/MEMORY.md, sonra docs/DELIVERY-PLAN.md ve docs/SDD.md bölüm 16 dosyalarını oku.

Sıradaki tek iş birimini belirle: en erken tamamlanmamış slice, ve onun içinde tek başına
tamamlanıp doğrulanabilecek en küçük parça.

Sadece şunu yazdır:
  SLICE: <id>
  UNIT: <tek satır>
  REASON: <tek satır, neden sırada bu>
  BLOCKED_BY: <madde kimlikleri veya none>

BLOCKED_BY "none" değilse dur. Meşgul kalmak için başka bir birim seçme.
```

2.3 Aşama 2 — spec yaz

```
docs/specs/SPEC-TEMPLATE.md dosyasından docs/specs/SPEC-<nnn>-<slug>.md oluştur:
  <aşama 1'deki UNIT>

Her bölümü doldur. Özellikle:
- Files: tam yollar; bu listenin dışına dokunulmayacak
- Verification: geçmesi gereken komutlar ve her birinin neyi kanıtladığı
- Gate: yedi sorunun hepsini dürüstçe yanıtla

Gate'i yanıtlamadan önce FILES TABLOSUNU YÜRÜ. Üç döngü, tam görünüp tam olmayan tablolara
harcandı. Oluşturacağın her dosya için:

1. Hangi assembly'de duruyor, hangi assembly'ler ona referans vermeli? Bu referansların her biri
   bir `.csproj` değişikliğidir - listele. docs/SDD.md bölüm 7.1'deki bağımlılık grafiğinde dışa
   dönük bir referans eksik satır değil, yanlış tasarımdır: dur ve escalate et.
2. Tip bir assembly sınırını geçiyor mu? O zaman public olur, kendi dosyası gerekir, ve tüketenin
   görmeye izinli olduğu bir abstraction lazım. O dosyayı da adlandır.
3. Onu gerçekten çağıran bir şey var mı? ValidationBehaviour'ı olmayan validator, repository'si
   olmayan handler, kimsenin çağırmadığı registration - hepsi derlenip hiçbir şey yapmayan dosya.
4. Henüz pinlenmemiş bir CLI, tool manifest veya paket gerekiyor mu? Bu gate 2'dir.
5. Dört corePackage varsaydığın şeyi içeriyor mu? Boş olabilirler. Varsayma, bak.

Cevapları spec'e yaz. Bu şekilde yürünmemiş bir Files tablosu bitmemiştir, ve onaylanması bir
```

```
insanın turunu çöpe atar.

Herhangi bir gate cevabı "yes" ise: status draft, escalation alanına hangi gate'in tetiklendiğini
ve nedenini yaz, dur. Devam etme.

Yedisi de "no" ise: tier A, status approved, approved_by agent.

Sadece spec yolunu ve status'unu yazdır.
```

2.4 Aşama 3 — uygula

```
docs/specs/SPEC-<nnn>-<slug>.md dosyasını birebir uygula.

Katı kurallar:
- spec'in Files tablosu dışındaki hiçbir dosyayı değiştirme
- spec'in yanlış olduğunu fark edersen status'u blocked yap, escalation'a nedenini yaz, dur.
  Farklı bir tasarımı doğaçlama yapma.
- AGENTS.md ve işaret ettiği standartlara uy
- bir şeyi geçirmek için analyzer severity düşürme, test silme, test skip etme

İlk düzenlemeden önce mevcut daldan feat/spec-<nnn>-<slug> dalını aç.
Başlarken status'u in-progress yap.
Değiştirdiğin dosyaların listesini yazdır.
```

2.5 Aşama 4 — doğrula ve kendi kendini tamir et

```
docs/specs/SPEC-<nnn>-<slug>.md içindeki Verification komutlarını çalıştır.

Hepsi geçerse: spec'in "Done when" listesini tamamla, status'u done yap, şunu yazdır
  RESULT: pass
ve dur.

Bir şey başarısız olursa en fazla 3 tamir döngüsü dene. Her döngüde:
1. Hatanın en kısa belirleyici satırını alıntıla.
2. Sınıflandır: compile error | test assertion | analyzer violation | environment.
3. Nedeni düzelt, sadece spec'in Files tablosu içinde.
4. Başarısız komutu tekrar çalıştır.

"Düzeltme" sayılmayan, yasak olanlar: test silmek veya skip etmek, assertion zayıflatmak,
.editorconfig severity düşürmek, gerekçe yorumu olmadan pragma suppression eklemek, spec'in Files
tablosunu genişletmek.

3 döngü sonunda hâlâ hatalıysa veya kalan tek çözüm yasak listedeyse:
  status'u blocked yap
  escalation'a hatayı ve denediğin düzeltmeleri yaz
  Elindekini spec'in dalına `wip(spec-<nnn>): ...` mesajıyla COMMIT et; working tree temiz kalsın
    ve sonraki döngü devam edebilsin. Blocked spec'i push etme.
  RESULT: blocked yazdır
  dur.

Bloğu commit etmek başarı iddiası değildir. Kanıt kirli ağaçta değil git'te durur, ve kirli ağaçta
başlamayı reddeden sürücü, insan ortalığı toplamadan tekrar çalışabilir.

Environment hataları (Docker kapalı, ağ yok) asla kod problemi değildir. RESULT: environment
```

```
yazdır ve hemen dur.
```

2.6 Aşama 5 — kapanış

```
Az önce tamamlanan spec için:
- docs/SDD.md bölüm 16 statüsünü dürüstçe güncelle (kod varsa "done")
- migration oluşturulduysa brain/migrations/migrations-index.md satırı ekle
- düzeltilmemiş güvenlik bulgusu varsa docs/owasp/THREAT-REGISTER.md satırı ekle
- kalıcı bir şey öğrenildiyse brain/lessons/lessons-index.md satırı ekle
- endpoint değiştiyse docs/api/openapi.json dosyasını yeniden üret
- işi spec'in dalına, spec kimliğini gövdede anan Conventional Commits mesajıyla commit et
- o dalı push et ve draft pull request aç:
  `gh pr create --draft --title "<type>(<scope>): <subject>" --body-file <yazdığın dosya>`
  Gövde şunları içerir: spec kimliği, ilerleyen subject gereksinimleri, doğrulama komutları ve
  sonuçları, OWASP checklist sonucu, ve inceleyenin elle kontrol etmesi gerekenler.
  Ready for review yapma. Merge etme.
- 5 satırlık özet yazdır: birim, değişen dosyalar, eklenen testler, ilerleyen gereksinimler, sıradaki
```

2.7 Durma koşulları

İnsansız koşu bunlardan herhangi birinde **durmak zorunda**. Öneri değil; birini geçen ajan incelenemez diff üretir.

Koşul	Neden
Herhangi bir spec gate cevabı “yes”	Tier B insan ister
3 tamir döngüsü tükendi	neden mekanik değil
Çözüm yasak bir eylem gerektiriyor	gate tam da bunu engellemek için var
Spec’in Files tablosu dışında bir dosya değişecek	kapsam kayması
Bir test silinecek, skip edilecek veya zayıflatılacak	kanıt testin kendisi
Docker veya ağ yok	ortam sorunu, kod değil
`.agent/STOP` dosyası var	insan ipi çekti
Başlangıçta working tree kirliydi	koşu ilgisiz değişiklikleri yakalar
Slice’ın ihtiyacı olan kimlik henüz yok	gate 5 escalate eder; ADR-0026
Bütçe aşıldı (koşu, token veya süre)	döngü sürücün uygular

2.8 Her insansız koşuya eklenecek guardrail bloğu

Bu bloğu her otonom promptun sonuna yapıştır:

```
Guardrails, mutlak:
- Sadece kendi uygulama kökünde (backend için MusicRoom/, client için MusicRoomApp/),
  docs/specs/ ve spec'in listelediği dosyalarda yaz. Diğer köke asla yazma.
- reference/ salt okunur. Düzenleme, oradan kopyalama.
- Asla secret okuma, yazdırma, commit etme. .env dosyasını açma.
- Asla .editorconfig severity düşürme, test silme, test skip etme.
- Kodu olmayan bir docs/SDD.md bölüm 16 satırını "done" yapma.
- ADR yazma veya değiştirme. Karar gerekiyorsa taslak çıkar ve dur.
- Bağımlılık ekleme veya yükseltme.
- Git: onaydan sonra tek bir feat/spec-<nnn>-<slug> dalı aç, doğrulama geçince tek commit at.
  Sadece RESULT: pass olduğunda o dalı push et ve gh pr create --draft ile DRAFT pull request aç.
  Spec başına bir PR.
- main'e veya spec'in kendi dalı dışında hiçbir dala push etme. --force, reset --hard, rebase,
  uzak dal veya tag silme yok. PR'ı ready for review yapma, merge etme, auto-merge açma, onaylama.
- blocked veya environment sonucunda hiçbir şey push edilmez.
```

```
- Kirli working tree ile koşuyu başlatma.
- .agent/STOP varsa hemen dur ve RESULT: halted yazdır.
```

2.9 Durduktan sonra devam

```
docs/specs/ dizinini oku ve status'u done olmayan her spec'i status ve escalation ile listele.
blocked veya draft olan her spec için hangi karar veya düzeltmeyi beklediğini tek satırda söyle.
Hiçbir şeyi değiştirme.
```

Bölüm 3 — Araca özel

3.1 Claude Code

Başka araçta karşılığı olmayan özellikler. Kullan; Bölüm 1'e taşıma.

Özellik	Ne için
Plan mode	ikiden fazla dosyaya dokunan her değişiklik. Dosya listesini sun, onay al, sonra çık
`/loop <aralık> <prompt>`	Bölüm 2 aşamalarını zamanlı sürmek. Aralığı boş bırakırsan model kendi hızını ayarlar
Subagent	sadece salt okunur fan-out: kod bulma, dizin haritalama. Uygulama ana thread'de kalsın ki diff tek yerde incelensin
`CLAUDE.md`	zaten `AGENTS.md`'ye işaret ediyor. İnce tut; ortak kurallar `AGENTS.md`'de
Hook (`.claude/settings.json`)	mekanik zorlama: `MusicRoom/` dışına yazmayı, `reference/` düzenlemeyi reddet
`/code-review`	yerleşik inceleme; yine de prompt 12 ile standartları besle

Otonom koşu, Claude Code

Terminalden, oturum içinden değil:

```
make agent-run            # kapsamlı izinler + canlı çıktı
make agent-run CYCLES=5
```

Düz `claude -p` **izin soramaz**, verilmemiş her yazma reddedilir ve döngü diskte hiçbir şey bırakmadan biter. `.agent/permissions.json` ve `--permission-prompts none` insansız koşunun gerçekten iş üretmesini sağlayan şey. O dosyadaki yol kuralları `Edit(...)` olmalı; `Write(yol)` kuralları sessizce yok sayılıyor.

İnteraktif oturum içinden:

```
/loop AGENTS.md dosyasını oku. docs/prompt-library/PROMPTS.md Bölüm 2'nin tam bir döngüsünü
çalıştır: aşama 1 seç, aşama 2 spec, aşama 3 uygula, aşama 4 doğrula ve tamir et, aşama 5 kapanış.
Bölüm 2.8'deki guardrail bloğunu uygula. Bölüm 2.7'deki her durma koşuluna uy.
.agent/STOP varsa döngüyü hemen durdur.
```

Yazma sınırını zorlayan hook

`.claude/settings.json` — mekanik, yani modelin yanlış okuduğu bir promptu atlatamaz:

```
{
  "hooks": {
```

```
    "PreToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "jq -r '.tool_input.file_path' | grep -qE '^(.*/)?(reference)/' && { echo 'refere
          }
        ]
      }
    ]
  }
}
```

3.2 Gemini CLI

- `GEMINI.md` otomatik okunuyor; `AGENTS.md`'ye işaret ediyor.
- Gemini'nin geniş context penceresi genişlemesine okumaya davet ediyor. Direnç göster — Brain tam da dört küçük dosyanın taramanın yerine geçmesi için var. İlk promptta "workspace'i tarama" ifadesini açıkça söyle.
- İnteraktif olmayan koşular promptu stdin'den alıyor; Bölüm 4 sürücüsü bunu kullanıyor.
- Yerleşik loop yok. Bölüm 4 sürücü script'ini kullan.

3.3 Cursor, Copilot, Aider, Codex

- Cursor: vazgeçilmezleri `.cursor/rules/` altına kısa ve her zaman açık bir kural olarak yansıt: "`AGENTS.md` oku". İçeriği kopyalama.
- Copilot: `AGENTS.md`'yi kendiliğinden okumaz. Sohbet başında prompt 1'i yapıştır.
- Aider: komut satırında `--read AGENTS.md brain/MEMORY.md`. Dosya kapsamına iyi uyuyor, Bölüm 2 aşama 3 için uygun.
- Dosya kapsamı olmayan araçlar: prompt 2'yi kullan ve diff'i kabul etmeden önce kendin incele.

3.4 CI içinden herhangi bir ajan

CI ajanlarını **salt okunur** tut. Kod düzenleyen bir pipeline, kırmızı build'i hareketli hedefe çevirir. İyi kullanım: prompt 12 çıktısını PR yorumu yapmak, prompt 15'i zamanlı çalıştırmak.

Bölüm 4 — Snippet'lar

4.1 Araç bağımsız sürücü

`.agent/run.sh` — depoda hazır. İnteraktif olmayan CLI'ı olan her ajanla çalışır.

```
AGENT_CMD="claude -p" ./.agent/run.sh 5
AGENT_CMD="gemini -p" ./.agent/run.sh 3
```

İpi çek:

```
touch .agent/STOP      # mevcut döngüden sonra dur
rm .agent/STOP         # devam et
```

Sürücü her döngüde `RESULT:` satırını okur ve `pass` dışındaki her sonuçta durur. Tanınmayan sonuçta güvenlik gereği durur.

4.2 Ajanın geçmesi gereken yerel kapı

`make agent-check` — CI'ın çalıştırdığı kapının aynısı, böylece tamir döngüsü gerçek hedefe yakınsar.

```
agent-check:
    dotnet format --verify-no-changes
    dotnet build -warnaserror
    dotnet test
    dotnet list package --vulnerable --include-transitive
```

4.3 Sonuç envelope'u

Bir script'in tükettiği her prompt tek makine-okunur satırla bitmeli:

`RESULT: pass | blocked | environment | halted | nothing-to-do`

Sonrasında hiçbir şey. Düz yazı ayrıştıran sürücü yanlış ateşler.

4.4 Spec durum taraması

```
grep -l '^status:' docs/specs/SPEC-*.md 2>/dev/null \
  | xargs -r awk '/^id:|^status:|^escalation:/{printf "%s ", $0} FNR==1{printf "\n%s: ", FILENAME}'
```

4.5 Gecelik Brain sağlık kontrolü

Salt okunur, zamanlanması güvenli:

```
AGENT_CMD="claude -p" $AGENT_CMD "$(sed -n '/## 15. Brain health check/,/^---/p' \
  docs/prompt-library/PROMPTS.md)"
```

Bölüm 5 — Anti-pattern'ler

Yapma	Neden	Yerine
"Repoyu tara ve oylamayı yaz"	her şeyi yeniden okur, verilmiş kararları yok sayar	prompt 1, sonra 2
"reference/nArchitecture'daki deseni kopyala"	salt okunur, net6, kopyalamak subject IV.1'i ihlal eder	prompt 2 — deseni oku, kendin yaz
"Testleri geçir yeter"	hatanın etrafına şekillenmiş test üretir	prompt 6
"Mimariyi sen tasarla"	subject bölüm II: vermediğin kararı savunamazsın	prompt 7 — sen karar ver, ajan kaydetsin
Prompta connection string veya key yapıştırmak	değer makineyi terk ettiği anda yanmıştır	`.env` anahtar adını referans ver
"Sence en iyisi neyse ADR yaz"	savunulmayan karar, açık karardan kötüdür	önce karar, sonra prompt 7
Sınırsız otonom döngü	incelenemez diff, tükenen bütçe	Bölüm 2.7 durma koşulları ve sürücünün döngü sınırı
Ajanın Tier B spec'i kendi onaylaması	gate bu değişikliklerin geri alınması pahalı olduğu için var	escalate; yedi soru tavsiye değil
CI'dan kod düzenleyen döngü	kırmızı build hareketli hedefe döner	CI ajanlarını salt okunur tut
" bitti mi?" sorusunu plandan cevaplamak	docs/SDD.md bölüm 16 koddan öne kaçar	prompt 16 — hüküm path:line göstermeli

---

İlgili

[[prompt-library]] (kanonik İngilizce) · `../../AGENTS.md` · [[specs]] · [[agent-runner]] · [[user-manual]] · [[sdd]] · [[delivery-plan]] · [[owasp-index]] · [[MEMORY]]